HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG IoT GIÁM SÁT NHIỆT ĐỘ, ĐỘ ẨM, ÁNH SÁNG VÀ ĐIỀU KHIỂN THIẾT BỊ

Sinh viên thực hiện: Lê Minh Trí

Mã sinh viên: B23DCCN839

Môn học: IoT và Ứng dụng

Giảng viên hướng dẫn: Nguyễn Quốc Uy

Hà Nội – 2026

MỤC LỤC

Chương 1. Giới thiệu đề tài

1.1. Đặt vấn đề

1.2. Lý do chọn đề tài

1.3. Mục tiêu đề tài

1.4. Phạm vi nghiên cứu

Chương 2. Thiết kế tổng quan

2.1. Kiến trúc hệ thống tổng thể

2.2. Thiết bị phần cứng và giao tiếp IoT

2.3. Công nghệ phần mềm

2.4. Các luồng hoạt động chính

2.5. Thiết kế giao diện người dùng

Chương 3. Thiết kế chi tiết

3.1. Thiết kế cơ sở dữ liệu

3.2. Đặc tả Use Case

3.3. Đặc tả REST API

Chương 4. Kết quả triển khai và kiểm thử

4.1. Công nghệ sử dụng

4.2. Kết quả triển khai Backend

4.3. Kết quả triển khai Frontend

4.4. Giao diện hệ thống

4.5. Kiểm thử và đánh giá

Chương 5. Kết luận

5.1. Kết quả đạt được

5.2. Hạn chế và hướng phát triển

Lưu ý: Mục lục được trình bày theo cấu trúc báo cáo mẫu. Có thể cập nhật số trang tự động trong Word sau khi mở tài liệu.

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

Internet of Things (IoT) cho phép các thiết bị vật lý thu thập, trao đổi dữ liệu và nhận lệnh điều khiển thông qua mạng. Trong môi trường phòng ở hoặc phòng làm việc, các thông số như nhiệt độ, độ ẩm và cường độ ánh sáng có ảnh hưởng trực tiếp đến sự thoải mái và điều kiện vận hành.

Nếu chỉ kiểm tra bằng phương pháp thủ công, người dùng khó theo dõi liên tục sự thay đổi của môi trường và khó lưu lại lịch sử để đối chiếu. Vì vậy, cần xây dựng một hệ thống có khả năng thu thập dữ liệu từ cảm biến, truyền dữ liệu về máy chủ, lưu trữ và cung cấp giao diện Web để giám sát cũng như điều khiển thiết bị từ xa.

1.2. Lý do chọn đề tài

Đề tài "Xây dựng hệ thống IoT giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển thiết bị" được lựa chọn nhằm vận dụng kiến thức về lập trình Backend, cơ sở dữ liệu, giao tiếp mạng và phát triển giao diện Web vào một bài toán thực tế.

Thông qua đề tài, sinh viên có cơ hội tìm hiểu mô hình giao tiếp MQTT giữa thiết bị IoT và máy chủ, xây dựng REST API cho các chức năng nghiệp vụ, tổ chức dữ liệu bằng MySQL và thiết kế các màn hình theo dõi trực quan.

1.3. Mục tiêu đề tài

- Thu thập ba thông số môi trường: nhiệt độ, độ ẩm và ánh sáng từ thiết bị IoT.
- Truyền dữ liệu cảm biến về Backend thông qua MQTT và lưu trữ vào cơ sở dữ liệu.
- Xây dựng giao diện Web hiển thị số liệu, biểu đồ, danh sách dữ liệu cảm biến và lịch sử điều khiển.
- Cho phép người dùng gửi lệnh bật/tắt hai đèn LED thông qua Backend; ghi nhận kết quả dựa trên phản hồi từ thiết bị.
- Cung cấp các REST API phục vụ truy vấn dữ liệu cảm biến, trạng thái thiết bị, lịch sử điều khiển và thông tin tài khoản.

1.4. Phạm vi nghiên cứu

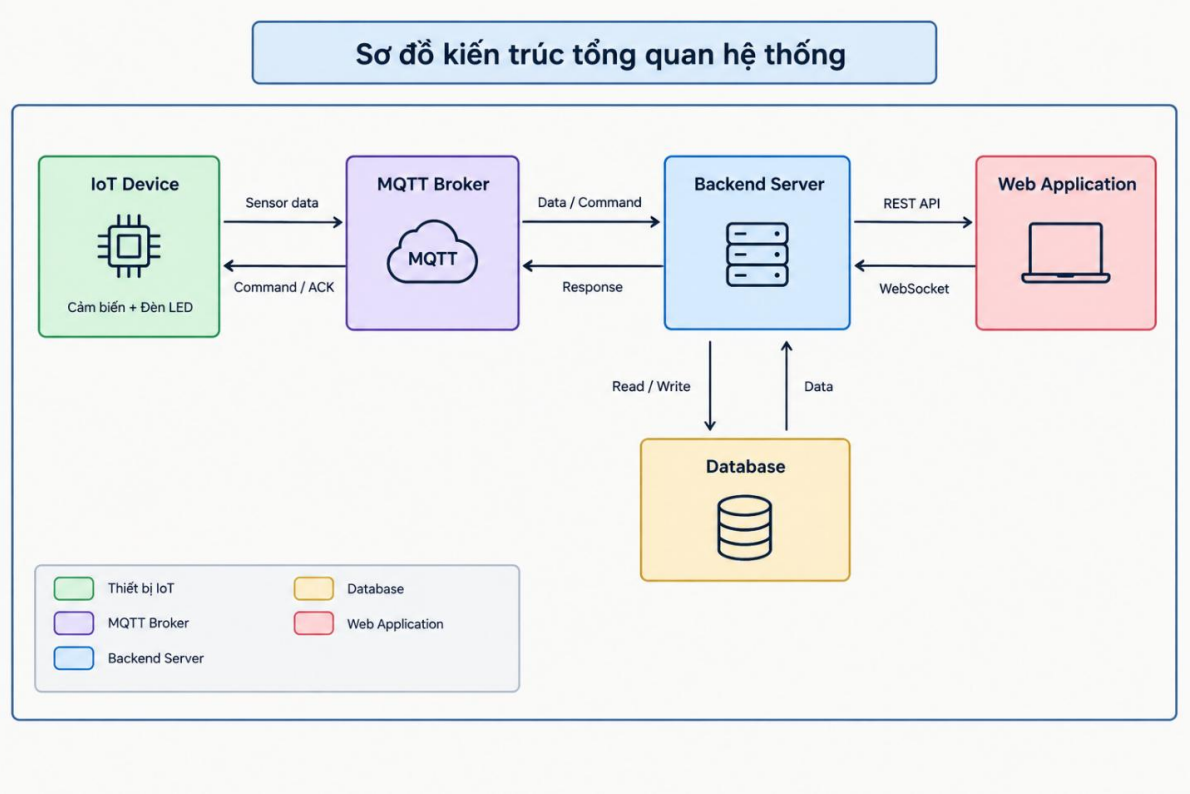
Hệ thống tập trung vào mô hình giám sát môi trường trong phạm vi một không gian phòng và điều khiển hai thiết bị đèn LED. Phần mềm gồm Backend Spring Boot, cơ sở dữ liệu MySQL và giao diện Web tĩnh được phục vụ từ ứng dụng.

Tài liệu SRS mô tả WebSocket cho cập nhật thời gian thực. Tuy nhiên, trong source code hiện tại chưa thấy triển khai WebSocket; phần giao diện Dashboard, bảng cảm biến và lịch sử vẫn còn dữ liệu mẫu/logic mô phỏng ở một số màn hình. Báo cáo phân biệt rõ phần yêu cầu thiết kế và phần đã thể hiện trong mã nguồn, không xem các chức năng mô phỏng là chức năng tích hợp hoàn chỉnh.

CHƯƠNG 2: THIẾT KẾ TỔNG QUAN

2.1. Kiến trúc hệ thống tổng thể

Theo tài liệu SRS, hệ thống gồm năm thành phần chính: IoT Device, MQTT Broker, Backend Server, Database và Web Application. Thiết bị IoT thu thập dữ liệu cảm biến và thực thi lệnh điều khiển; MQTT Broker trung chuyển bản tin; Backend xử lý nghiệp vụ và truy cập cơ sở dữ liệu; Web Application cung cấp giao diện cho người dùng.



Hình 2.1. Sơ đồ kiến trúc tổng quan hệ thống

2.2. Thiết bị phần cứng và giao tiếp IoT

Thiết bị IoT đảm nhiệm việc đọc dữ liệu từ cảm biến nhiệt độ, độ ẩm và ánh sáng, đồng thời điều khiển hai đèn LED. Trong mã Backend, topic trạng thái có tên iot/esp8266/status, cho thấy hệ thống có tích hợp thiết bị ESP8266. Loại cảm biến cụ thể và sơ đồ chân phần cứng không được mô tả trong source code Backend được cung cấp nên không khẳng định model linh kiện.

MQTT được sử dụng để trao đổi bản tin giữa thiết bị và Backend. Các topic được khai báo trong MqttService như sau:

Topic MQTT	Chiều truyền	Mục đích
data-sensor	ESP → Backend	Gửi dữ liệu temperature, humidity, light.

data-feedback	ESP → Backend	Gửi phản hồi kết quả điều khiển, gồm deviceId, action và status.
device-control	Backend → ESP	Gửi lệnh bật/tắt LED.
iot/esp8266/status	ESP → Backend	Thông báo trạng thái online/offline.

2.3. Công nghệ phần mềm

Thành phần	Công nghệ / thư viện	Vai trò
Backend	Java, Spring Boot	Xử lý nghiệp vụ và cung cấp REST API.
ORM	Spring Data JPA, Hibernate	Ánh xạ entity và thao tác dữ liệu.
Database	MySQL	Lưu tài khoản, thiết bị, dữ liệu cảm biến và lịch sử.
IoT messaging	Eclipse Paho MQTT Client	Kết nối MQTT Broker, subscribe và publish bản tin.
Frontend	HTML, CSS, JavaScript	Xây dựng các trang Dashboard, Sensors, History, Account.
Build / runtime	Maven, Spring Boot embedded server	Quản lý thư viện và khởi chạy ứng dụng.

2.4. Các luồng hoạt động chính

2.4.1. Luồng thu thập dữ liệu cảm biến

1. Thiết bị IoT đọc các giá trị nhiệt độ, độ ẩm và ánh sáng.
2. Thiết bị đóng gói dữ liệu và publish lên topic data-sensor.
3. Backend đã subscribe topic nhận bản tin MQTT và phân tích nội dung JSON.
4. MqttService cập nhật các giá trị mới nhất trong bộ nhớ ứng dụng.
5. Định kỳ theo cấu hình app.save-interval-seconds (mặc định 10 giây), Backend lưu ba bản ghi tương ứng vào bảng sensor_readings.
6. REST API cung cấp dữ liệu mới nhất, danh sách dữ liệu và dữ liệu tổng hợp theo giờ cho giao diện.

2.4.2. Luồng điều khiển thiết bị

7. Người dùng yêu cầu bật hoặc tắt LED_1/LED_2 trên giao diện.
8. Backend kiểm tra mã thiết bị và trạng thái kết nối của ESP.
9. Nếu thiết bị offline, Backend lưu lịch sử FAILED và trả thông báo lỗi.
10. Nếu thiết bị online, Backend tạo bản ghi PENDING, sau đó publish lệnh lên topic device-control.
11. Thiết bị thực thi lệnh và gửi bản tin phản hồi lên data-feedback.
12. MqttService tìm bản ghi PENDING tương ứng và cập nhật SUCCESS hoặc FAILED; nếu thành công, trạng thái thiết bị trong bảng devices được cập nhật.

13. DeviceController chờ phản hồi tối đa 5 giây. Nếu hết thời gian, thao tác được đánh dấu FAILED.

2.4.3. Luồng dữ liệu giữa Web và Backend

Web Application sử dụng REST API để lấy dữ liệu cảm biến, trạng thái thiết bị, lịch sử và thông tin tài khoản. SRS có yêu cầu WebSocket để đẩy dữ liệu theo thời gian thực; tuy nhiên, trong các tệp JavaScript hiện tại chưa có mã thiết lập WebSocket. Do đó, đây là yêu cầu thiết kế cần hoàn thiện ở giai đoạn tiếp theo.

2.5. Thiết kế giao diện người dùng

Theo SRS và các ảnh giao diện đính kèm, Web Application có bốn màn hình: Dashboard, Data Sensor, History và Account. Giao diện sử dụng thanh điều hướng bên trái, khu vực nội dung chính và các thẻ/bảng dữ liệu.

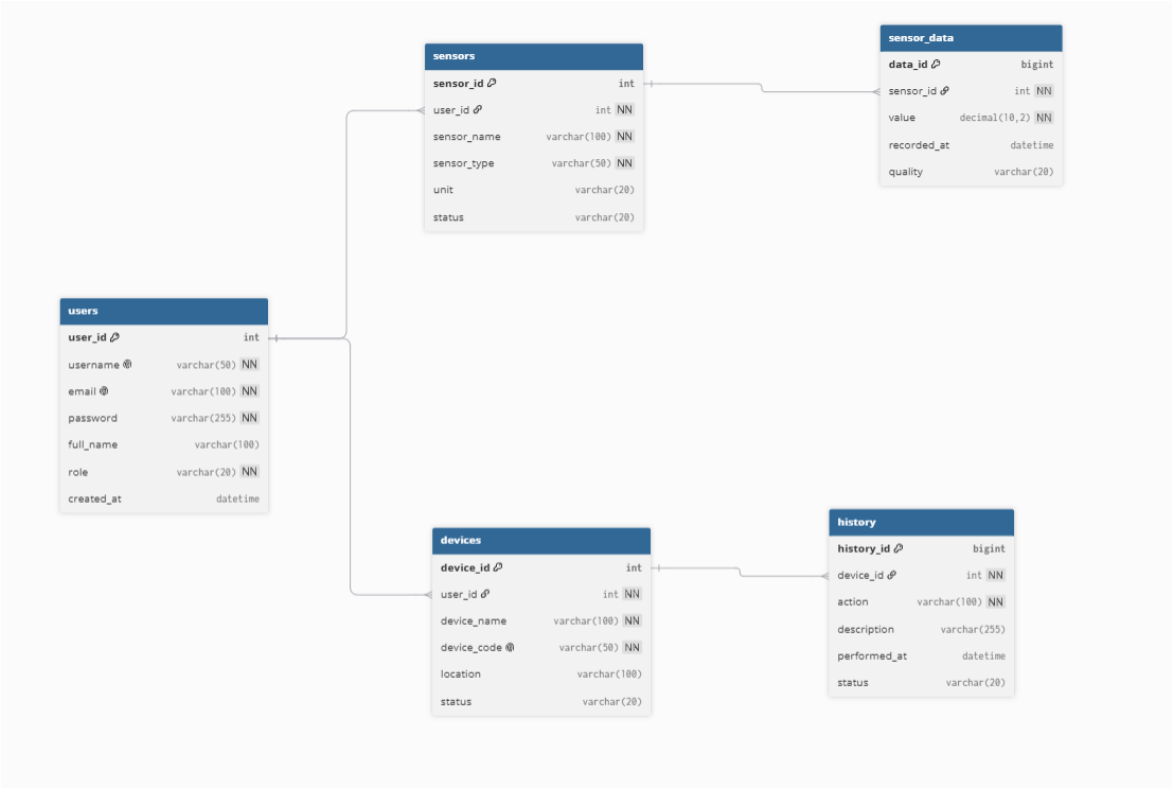
CHƯƠNG 3: THIẾT KẾ CHI TIẾT

3.1. Thiết kế cơ sở dữ liệu

Từ các lớp Entity trong source code, cơ sở dữ liệu sử dụng bốn bảng chính: accounts, devices, device_history và sensor_readings. Đây là cấu trúc được xác định từ mã nguồn hiện tại, khác với mô hình năm bảng trong báo cáo tham khảo.

Bảng	Trường chính	Ý nghĩa
accounts	id (PK), github, figma, postman, pdfName, pdfFile	Lưu liên kết tài nguyên và thông tin tệp PDF của tài khoản.
devices	code (PK), name, state	Lưu mã thiết bị, tên thiết bị và trạng thái bật/tắt.
device_history	id (PK), deviceCode, action, status, createdAt	Lưu lịch sử lệnh điều khiển, trạng thái PENDING/SUCCESS/FAILED và thời gian.
sensor_readings	id (PK), type, reading, unit, status, createdAt	Lưu giá trị cảm biến, đơn vị, tình trạng bản ghi và thời điểm đo.

Các entity được ánh xạ bằng Jakarta Persistence. Khóa chính của sensor_readings và device_history được sinh tự động; devices dùng mã thiết bị dạng chuỗi làm khóa chính; accounts hiện sử dụng một bản ghi có id = 1.



Hình 3.1. Sơ đồ ERD theo tài liệu SRS

Lưu ý: Sơ đồ ERD trong SRS cần được rà soát để bảo đảm khớp với bốn entity thực tế trong source code. Khi cập nhật bản ERD cuối cùng, nên thể hiện đúng tên bảng và thuộc tính như phần mô tả trên.

3.2. Đặc tả các Use Case

SRS xác định bốn Use Case chính với tác nhân là Người dùng (User).

Mã UC	Tên Use Case	Mục đích
UC-01	Xem tổng quan hệ thống (Dashboard)	Theo dõi nhiệt độ, độ ẩm, ánh sáng, biểu đồ và trạng thái hai đèn.
UC-02	Điều khiển bật/tắt đèn	Gửi lệnh bật/tắt hai đèn và nhận kết quả thực thi.
UC-03	Tra cứu dữ liệu cảm biến	Xem dữ liệu cảm biến dạng bảng, tìm kiếm, sắp xếp và phân trang.
UC-04	Xem lịch sử điều khiển thiết bị	Tra cứu các thao tác điều khiển, kết quả và thời gian.

3.2.1. UC-01: Xem tổng quan hệ thống (Dashboard)

Tác nhân: Người dùng (User).

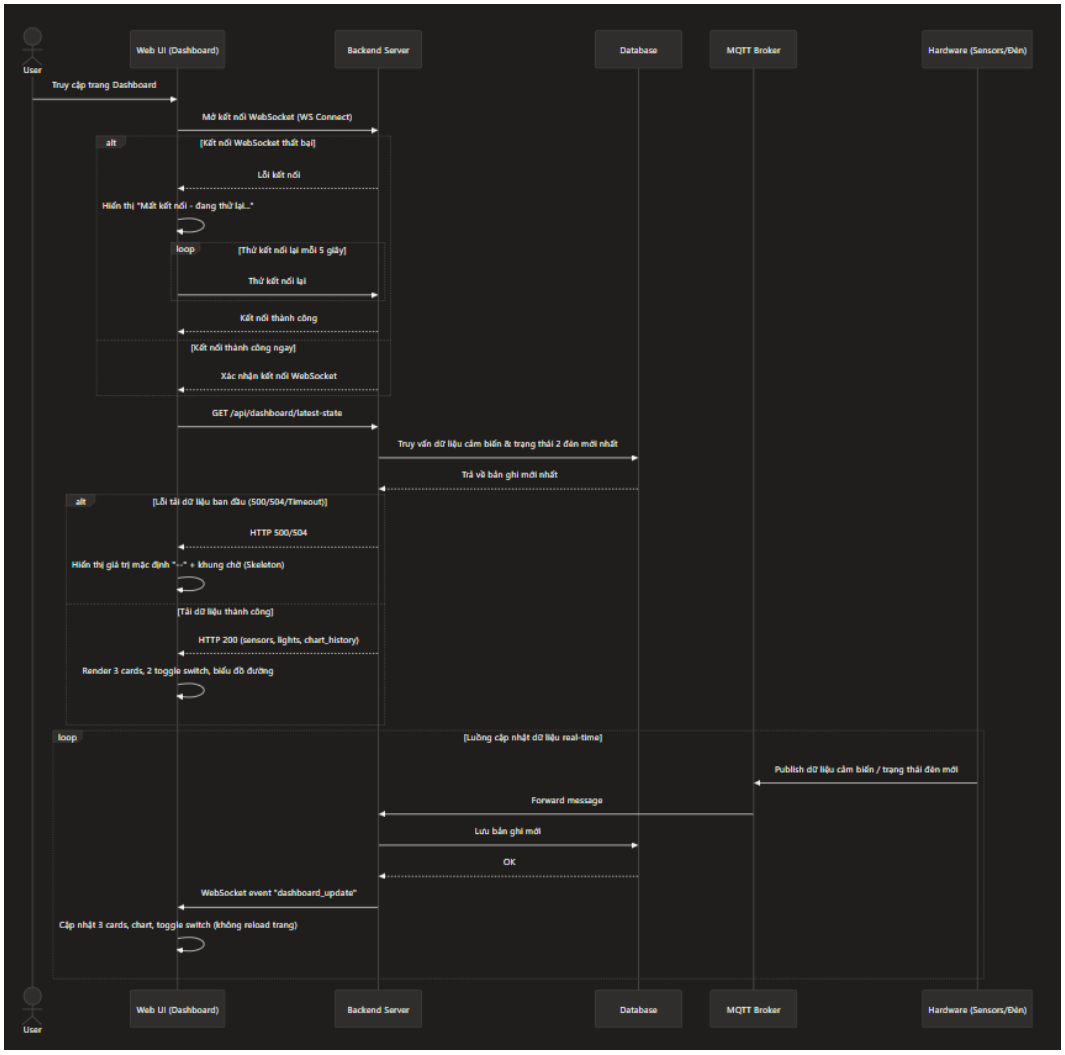
Mô tả: Người dùng theo dõi các chỉ số nhiệt độ, độ ẩm, ánh sáng và trạng thái hai đèn LED trên màn hình Dashboard; SRS đặt mục tiêu cập nhật dữ liệu theo thời gian thực.

Điều kiện tiên quyết: Người dùng đã truy cập hệ thống; Backend và MQTT Broker đang hoạt động; thiết bị IoT có thể gửi dữ liệu.

Luồng chính:

14. Người dùng mở trang Dashboard.
15. Hệ thống lấy dữ liệu cảm biến mới nhất và trạng thái thiết bị từ Backend.
16. Giao diện hiển thị ba thẻ thông số, biểu đồ và hai công tắc điều khiển.
17. Khi có dữ liệu mới, giao diện cần cập nhật mà không yêu cầu tải lại trang.

Luồng ngoại lệ: Khi API lỗi, giao diện cần hiển thị trạng thái không có dữ liệu và cho phép thử lại. Khi WebSocket mất kết nối, hệ thống cần cảnh báo và kết nối lại theo yêu cầu SRS. Chức năng WebSocket chưa được tìm thấy trong source hiện tại.



Hình 3.2.1. Sequence Diagram – UC-01

3.2.2. UC-02: Điều khiển bật/tắt đèn

Tác nhân: Người dùng (User). Mục đích: điều khiển LED1 hoặc LED2 từ giao diện Web và ghi nhận kết quả dựa trên phản hồi thiết bị.

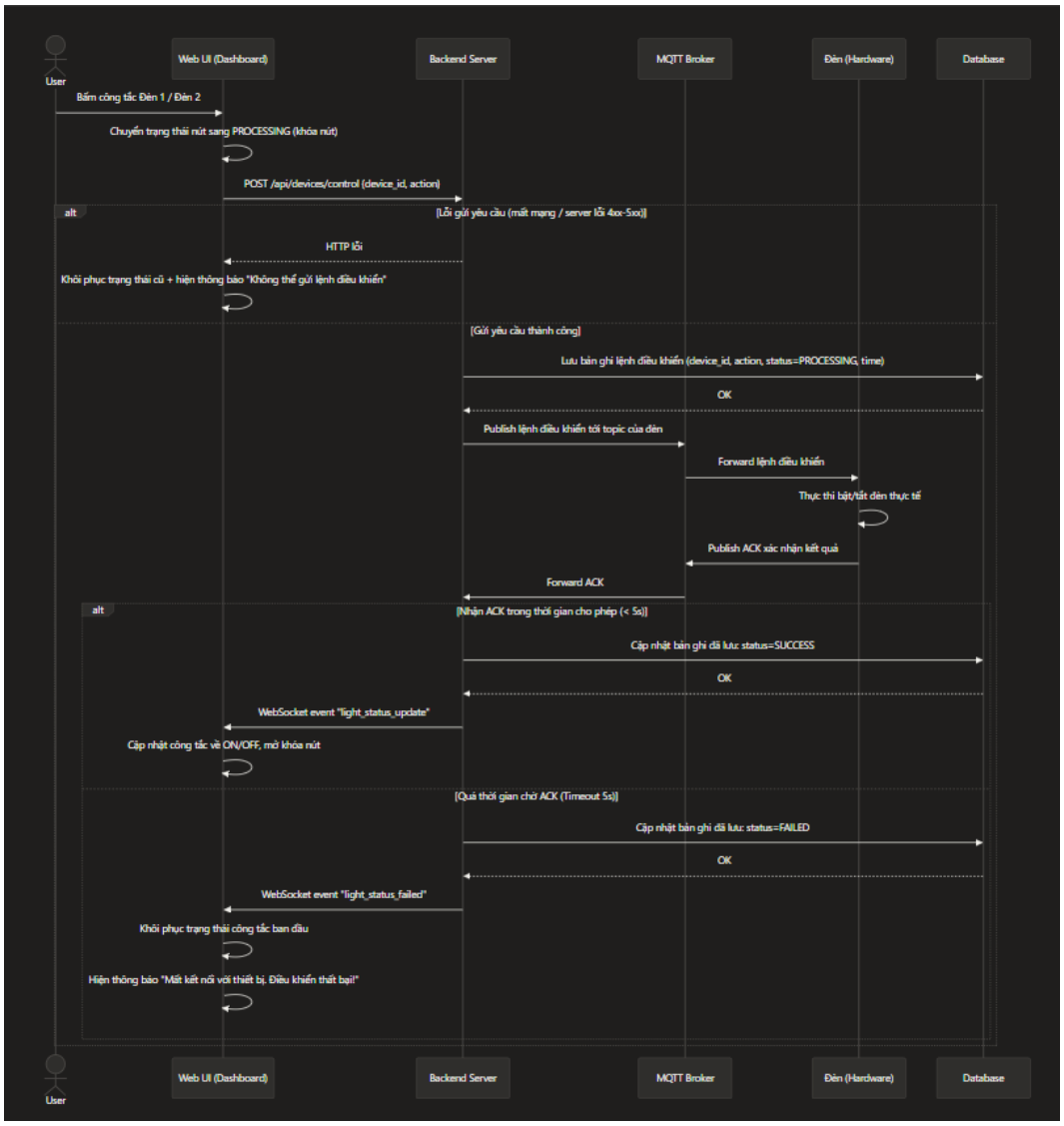
Điều kiện tiên quyết: Người dùng đang ở Dashboard; thiết bị và Backend có kết nối MQTT.

Luồng chính:

18. Người dùng thao tác công tắc của một đèn.
19. Backend nhận yêu cầu điều khiển, kiểm tra mã thiết bị và khả năng kết nối.
20. Backend ghi lịch sử ở trạng thái PENDING rồi publish lệnh MQTT.
21. Thiết bị thực thi và gửi phản hồi qua topic data-feedback.
22. Backend cập nhật lịch sử thành SUCCESS hoặc FAILED; nếu thành công, cập nhật trạng thái thiết bị.

23. Giao diện thông báo kết quả và hiển thị trạng thái tương ứng.

Luồng ngoại lệ: Thiết bị offline; không publish được lệnh; thiết bị trả về thất bại; hoặc không nhận phản hồi trong 5 giây. Các trường hợp này được ghi nhận FAILED.



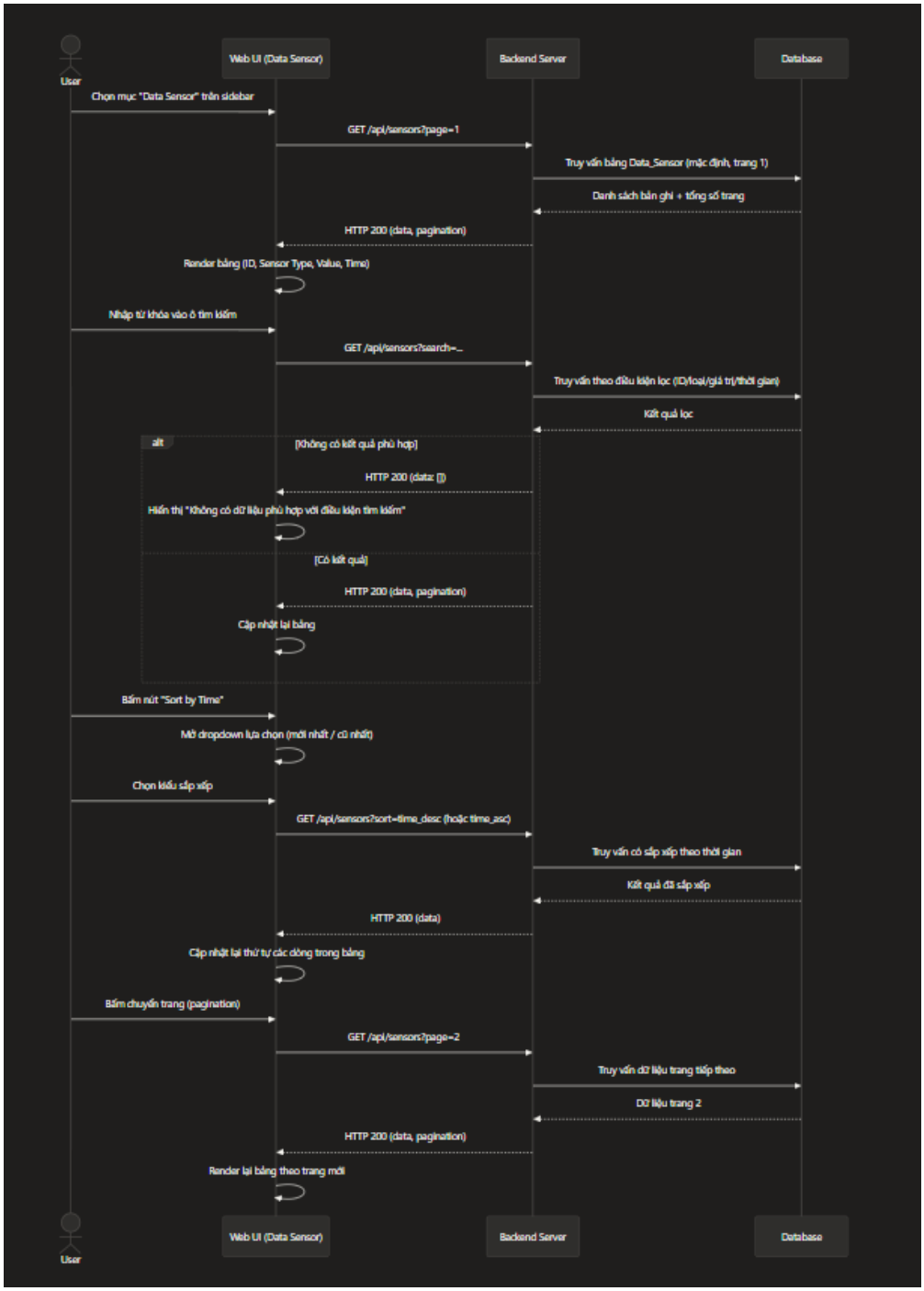
Hình 3.2.2. Sequence Diagram – UC-02

3.2.3. UC-03: Tra cứu dữ liệu cảm biến (Data Sensor)

Tác nhân: Người dùng (User). Mục đích: hiển thị các bản ghi cảm biến gồm ID, loại cảm biến, giá trị, trạng thái và thời gian; hỗ trợ tìm kiếm, lọc, sắp xếp và phân trang theo SRS.

Luồng chính: Người dùng mở Data Sensor; hệ thống lấy danh sách dữ liệu; giao diện hiển thị bảng; người dùng nhập điều kiện tìm kiếm hoặc chọn sắp xếp; bảng được cập nhật theo điều kiện.

Ngoại lệ: Không có bản ghi phù hợp thì hiển thị thông báo rỗng; nếu API lỗi thì hiển thị trạng thái lỗi và nút tải lại.



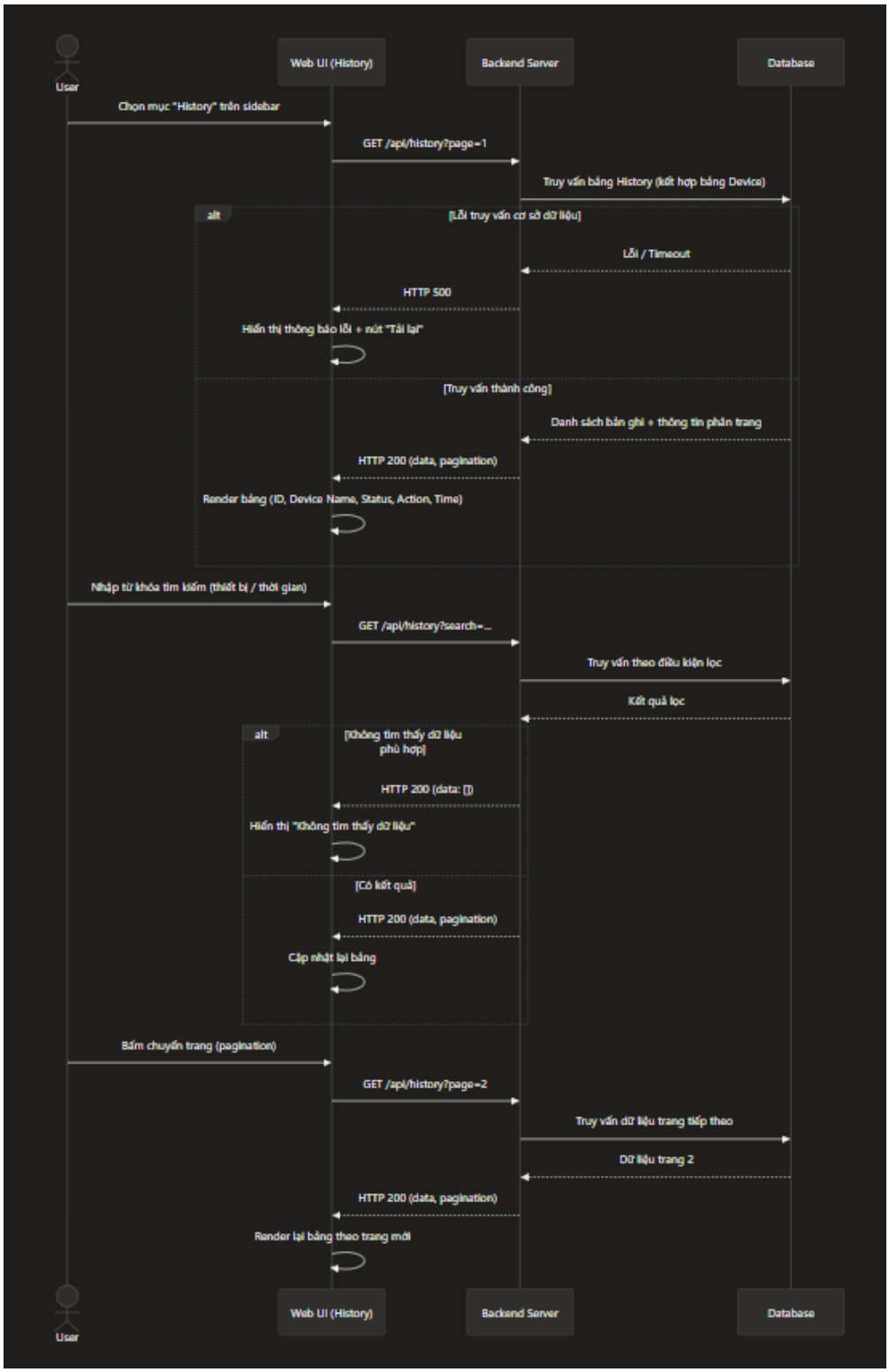
Hình 3.2.3. Sequence Diagram – UC-03

3.2.4. UC-04: Xem lịch sử điều khiển thiết bị (History)

Tác nhân: Người dùng (User). Mục đích: xem lại thiết bị được điều khiển, trạng thái thực thi, hành động và thời gian. SRS yêu cầu hỗ trợ tìm kiếm và phân trang.

Luồng chính: Người dùng mở History; hệ thống truy vấn danh sách lịch sử; giao diện hiển thị bảng gồm ID, Device Name, Status, Action và Time; người dùng tìm kiếm hoặc chuyển trang.

Ngoại lệ: Khi truy vấn lỗi, giao diện thông báo lỗi và cho phép tải lại; khi không có kết quả, hiển thị thông báo không tìm thấy dữ liệu.



Hình 3.2.4. Sequence Diagram – UC-04

3.3. Đặc tả REST API

Các API dưới đây được tổng hợp trực tiếp từ các lớp Controller trong source code. Base URL mặc định chạy tại cổng 8080.

Method	Endpoint	Chức năng
GET	/api/status	Kiểm tra trạng thái kết nối MQTT/thiết bị.
GET	/api/sensors/latest	Lấy các giá trị cảm biến mới nhất và cờ online/hasData.
GET	/api/sensors	Lấy tối đa 1000 bản ghi cảm biến mới nhất.
GET	/api/sensors/chart	Lấy dữ liệu trung bình theo giờ trong 12 giờ gần nhất.
GET	/api/devices	Lấy danh sách hai thiết bị và trạng thái hiện tại.
POST	/api/devices/{code}/control	Gửi lệnh bật/tắt thiết bị; body gồm on: true/false.
GET	/api/history	Lấy tối đa 1000 bản ghi lịch sử điều khiển mới nhất.
GET	/api/account	Lấy thông tin liên kết tài khoản.
PUT	/api/account	Cập nhật GitHub, Figma và Postman.
POST	/api/account/upload	Tải lên tệp PDF, giới hạn 10 MB.
GET	/api/account/pdf	Mở tệp PDF đã tải lên.

3.3.1. API lấy dữ liệu cảm biến mới nhất

Method: GET. Endpoint: /api/sensors/latest. Response chứa các trường online, hasData, light, temperature và humidity. Các giá trị được lưu tạm trong bộ nhớ MqttService và cập nhật khi nhận bản tin data-sensor.

3.3.2. API điều khiển thiết bị

Method: POST. Endpoint: /api/devices/{code}/control. Ví dụ code là LED1 hoặc LED2.

Thành phần	Ví dụ / mô tả
Request Body	{"on": true} để bật; {"on": false} để tắt.
Thành công	Response có success = true và message = Thành công.
Thất bại	Response có success = false và message mô tả thiết bị offline, lỗi gửi lệnh hoặc hết thời gian chờ.

3.3.3. API dữ liệu cảm biến và lịch sử

GET /api/sensors trả về danh sách bản ghi với id, type, value, status, time và ts. GET /api/history trả về id, device, status, action, time và ts. Trong source hiện tại, hai API trả về tối đa 1000 bản ghi mới nhất; việc tìm kiếm, sắp xếp và phân trang chủ yếu được xử lý ở JavaScript phía trình duyệt.

3.3.4. API tài khoản và tệp PDF

Nhóm /api/account hỗ trợ đọc/cập nhật các liên kết GitHub, Figma, Postman; tải lên PDF có kiểm tra phần mở rộng, dung lượng tối đa 10 MB và chữ ký tệp %PDF; đồng thời cung cấp endpoint mở tệp đã tải.

CHƯƠNG 4: KẾT QUẢ TRIỂN KHAI VÀ KIỂM THỬ

4.1. Công nghệ sử dụng

Phần Backend được tổ chức theo các package controller, entity, repository, mqtt và util. Frontend gồm các trang HTML tĩnh, stylesheet CSS và các tệp JavaScript riêng theo từng màn hình.

Nhóm	Thành phần đã có trong source
Controller	AccountController, DeviceController, HistoryController, SensorController.
Entity	Account, Device, DeviceHistory, SensorReading.
Repository	AccountRepository, DeviceRepository, DeviceHistoryRepository, SensorReadingRepository.
MQTT	MqttService dùng Eclipse Paho; tự kiểm tra kết nối theo chu kỳ 5 giây.
Frontend pages	index.html, sensors.html, history.html, account.html.
Frontend scripts	common.js, dashboard.js, sensors.js, history.js, table.js, account.js.

4.2. Kết quả triển khai Backend

Backend được xây dựng bằng Spring Boot và cấu hình chạy tại cổng 8080. Ứng dụng khởi tạo sẵn hai thiết bị LED1, LED2 và một bản ghi tài khoản nếu chưa tồn tại. Hibernate được cấu hình ddl-auto=update để cập nhật cấu trúc bảng theo entity.

MqttService sử dụng Eclipse Paho để kết nối broker được cấu hình trong application.properties. Backend subscribe các topic data-sensor, data-feedback và iot/esp8266/status; đồng thời publish lệnh lên device-control.

Dữ liệu cảm biến nhận được được giữ trong bộ nhớ để phục vụ API latest. Cứ mỗi khoảng thời gian cấu hình (mặc định 10 giây), hệ thống lưu ba bản ghi cảm biến vào sensor_readings. Dữ liệu biểu đồ được tổng hợp trung bình theo giờ trong 12 giờ gần nhất.

4.3. Kết quả triển khai Frontend

Frontend hiện có bốn trang giao diện. common.js tạo sidebar, biểu tượng và thông báo Toast; table.js cung cấp thành phần bảng dùng chung với lọc, tìm kiếm, sắp xếp và phân trang phía trình duyệt.

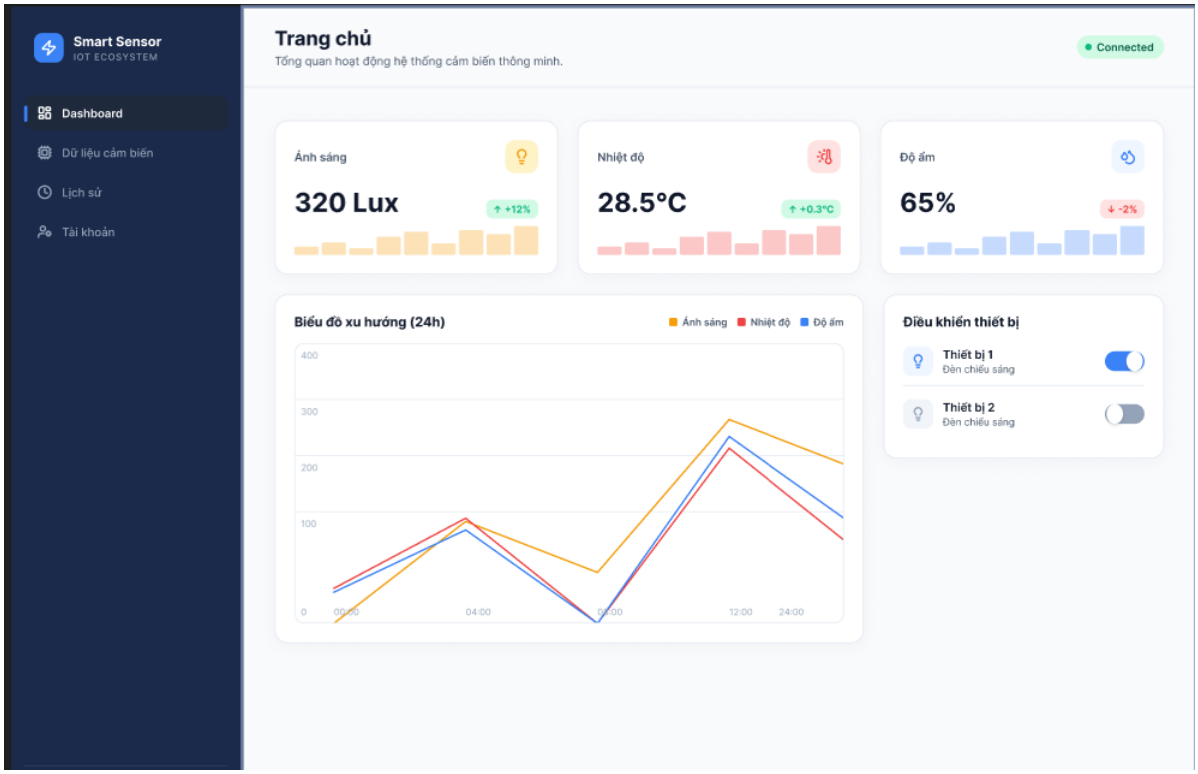
Cần lưu ý trạng thái tích hợp: dashboard.js hiện sử dụng dữ liệu biểu đồ khai báo sẵn và thao tác bật/tắt mô phỏng bằng setTimeout; sensors.js và history.js tạo dữ liệu mẫu trong JavaScript. account.js lưu các liên kết vào localStorage, trong khi Backend đã có API account tương ứng nhưng JavaScript chưa gọi các API đó. Vì vậy, giao diện và API Backend đã có các phần riêng, nhưng chưa được nối hoàn chỉnh ở tất cả màn hình.

4.4. Giao diện hệ thống

Các hình dưới đây là ảnh giao diện thiết kế được đưa vào tài liệu SRS. Chúng minh họa bố cục và chức năng dự kiến của ứng dụng.

4.4.1. Màn hình Dashboard

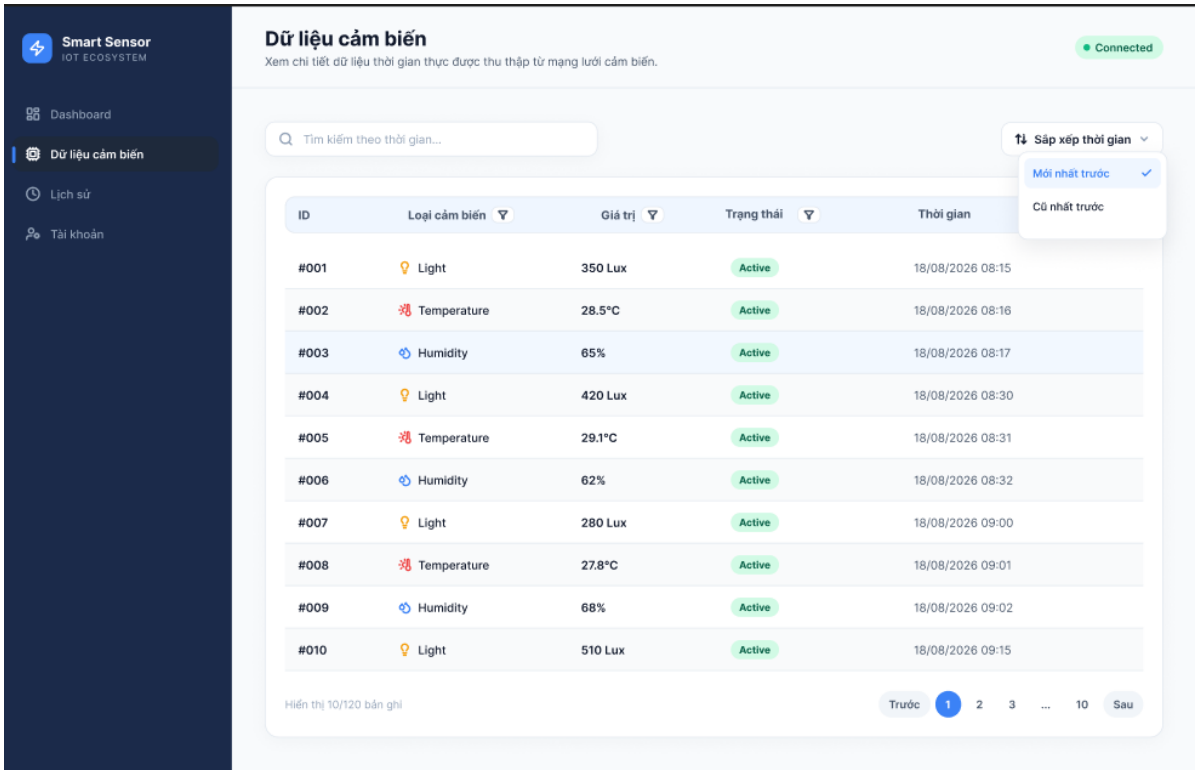
Dashboard trình bày ba thẻ chỉ số ánh sáng, nhiệt độ và độ ẩm; biểu đồ xu hướng; khu vực điều khiển hai đèn LED.



Hình 4.1. Giao diện Dashboard

4.4.2. Màn hình Data Sensor

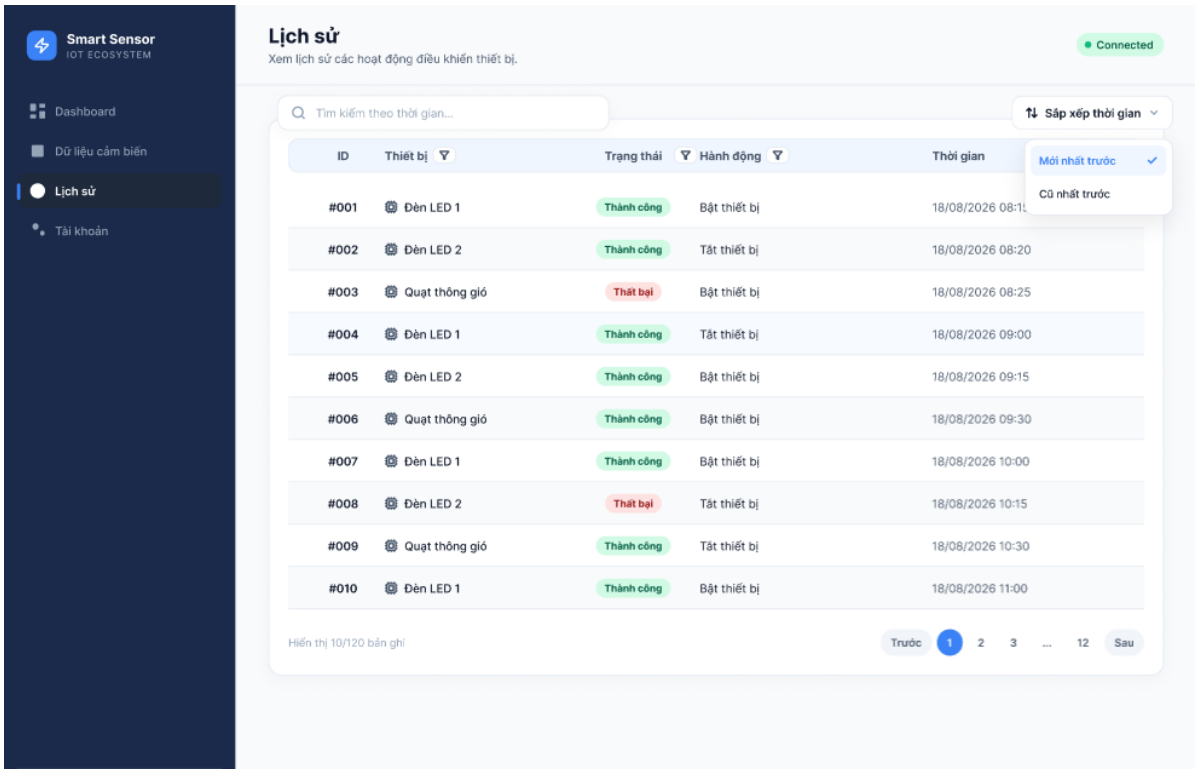
Màn hình Data Sensor hiển thị danh sách dữ liệu cảm biến dạng bảng. Tệp sensors.js hiện đang tạo dữ liệu minh họa, còn API GET /api/sensors đã được xây dựng ở Backend.



Hình 4.2. Giao diện Data Sensor

4.4.3. Màn hình History

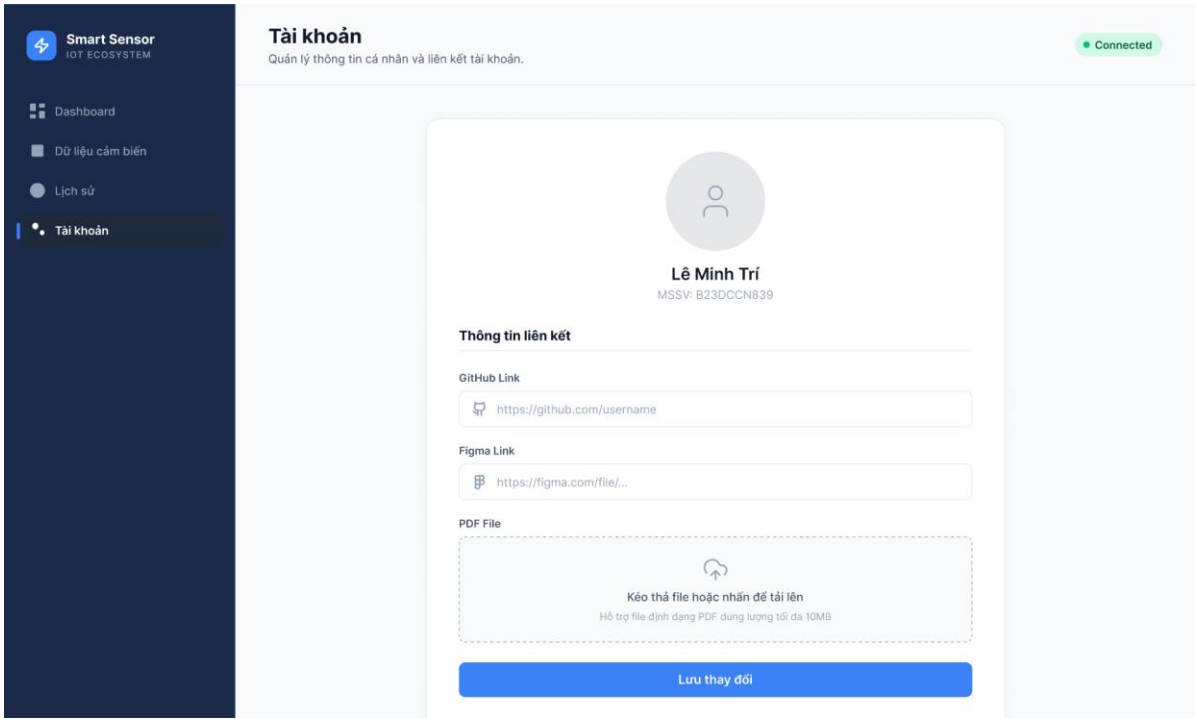
Màn hình History trình bày nhật ký các thao tác điều khiển, gồm thiết bị, trạng thái, hành động và thời gian. Tệp history.js hiện đang sử dụng dữ liệu mẫu.



Hình 4.3. Giao diện History

4.4.4. Màn hình Account

Màn hình Account hiển thị thông tin sinh viên, các trường liên kết GitHub/Figma và khu vực tải lên PDF. Backend đã có API đọc/cập nhật liên kết và upload PDF; phần JavaScript hiện tại vẫn lưu một số thông tin ở localStorage.



Hình 4.4. Giao diện Account

4.5. Kiểm thử và đánh giá

Phần kiểm thử được thiết kế theo hướng kiểm thử RESTful API bằng Postman. Các kịch bản dưới đây được xây dựng dựa trên endpoint và logic xử lý có trong các lớp Controller của Backend Spring Boot. Mỗi yêu cầu cần được gửi tới máy chủ đang chạy, sau đó đối chiếu HTTP status code và dữ liệu JSON trả về với kết quả mong đợi.

STT	Endpoint	Method	Kịch bản kiểm thử	Input	Expected	KQ
1	/api/status	GET	Kiểm tra trạng thái kết nối MQTT	Không có	HTTP 200; JSON có trường online kiểu boolean.	Pass
2	/api/sensors/latest	GET	Lấy dữ liệu cảm biến mới nhất	Không có	HTTP 200; trả về các trường online, hasData, light, temperature và humidity.	Pass
3	/api/sensors	GET	Lấy danh sách dữ liệu cảm biến	Không có	HTTP 200; trả về danh sách bản ghi gồm id, type, value, status, time và ts (tối đa 1000 bản ghi).	Pass
4	/api/sensors/chart	GET	Lấy dữ liệu tổng hợp để vẽ biểu đồ	Không có	HTTP 200; trả về labels, light, temperature và humidity; dữ liệu được tổng hợp theo giờ trong 12 giờ gần nhất.	Pass
5	/api/devices	GET	Lấy danh sách và trạng thái các thiết bị	Không có	HTTP 200; trả về danh sách thiết bị với code, name và on; dữ liệu lấy từ bảng devices.	Pass

STT	Endpoint	Method	Kịch bản kiểm thử	Input	Expected	KQ
6	/api/devices/LED1/control	POST	Gửi lệnh bật đèn LED1	{"on": true}	Nếu thiết bị online và phản hồi thành công: HTTP 200, success=true; trạng thái lịch sử được cập nhật SUCCESS. Nếu không nhận phản hồi trong 5 giây: success=false và ghi FAILED.	Pass
7	/api/devices/UNKNOWN/control	POST	Gửi lệnh tới mã thiết bị không tồn tại	{"on": true}	HTTP 404; không tìm thấy thiết bị tương ứng.	Pass
8	/api/devices/LED1/control	POST	Điều khiển khi ESP đang offline	{"on": false}	HTTP 200; JSON có success=false và thông báo thiết bị offline; lịch sử thao tác được lưu với trạng thái FAILED.	Pass
9	/api/history	GET	Lấy lịch sử điều khiển thiết bị	Không có	HTTP 200; trả về danh sách tối đa 1000 bản ghi mới nhất, gồm id, device, status, action, time và ts.	Pass
10	/api/account	GET	Lấy thông tin tài khoản	Không có	HTTP 200; trả về thông tin tài khoản cùng các trường liên kết và thông tin tệp PDF nếu đã có.	Pass
11	/api/account	PUT	Cập nhật các liên kết GitHub, Figma và Postman	{"github":"https://github.com/...","figma":"https://figma.com/...","postman":"https://..."}	HTTP 200; trả về thông tin tài khoản sau cập nhật và lưu các giá trị gửi lên.	Pass
12	/api/account/upload	POST	Tải lên tệp PDF hợp lệ	form-data: key=file; chọn tệp PDF hợp lệ, dung lượng không quá 10 MB	HTTP 200; trả về pdfName; tệp được lưu và thông tin tệp được cập nhật trong tài khoản.	Pass
13	/api/account/upload	POST	Tải lên tệp không hợp lệ	form-data: key=file; chọn tệp không phải PDF hoặc tệp vượt quá 10 MB	HTTP 400; trả về thông báo lỗi tương ứng: chỉ chấp nhận PDF, file vượt quá 10 MB hoặc PDF không hợp lệ.	Pass
14	/api/account/pdf	GET	Mở tệp PDF đã tải lên	Không có (thực hiện sau kịch bản upload PDF hợp lệ)	HTTP 200; Content-Type là application/pdf và nội dung phản hồi là tệp PDF. Nếu chưa có tệp hợp lệ, API trả HTTP 404.	Pass

Lưu ý: Các mã LED1/LED2 cần được đối chiếu với dữ liệu thực tế trong bảng devices. Đối với API điều khiển, kết quả phụ thuộc trạng thái kết nối MQTT và phản hồi của ESP8266; không thể khẳng định thành công chỉ dựa trên mã nguồn.

Bảng kiểm thử gồm 14 trường hợp, bao phủ các nhóm API trạng thái hệ thống, dữ liệu cảm biến, điều khiển thiết bị, lịch sử và tài khoản. Do chưa có Postman Collection hoặc ảnh/log kết quả thực thi được cung cấp kèm mã nguồn, cột Kết quả được để ở trạng thái "Chưa xác nhận". Sau khi thực thi trên Postman, cần cập nhật cột này theo kết quả thực tế; không nên kết luận 100% kiểm thử thành công nếu chưa có minh chứng chạy thử.

CHƯƠNG 5: KẾT LUẬN

5.1. Kết quả đạt được

- Xây dựng được cấu trúc Backend Spring Boot theo các lớp Controller, Entity, Repository và Service.
- Tích hợp MQTT Client để nhận dữ liệu cảm biến, nhận phản hồi điều khiển và gửi lệnh tới thiết bị.
- Thiết kế các entity và repository phục vụ lưu trữ tài khoản, thiết bị, dữ liệu cảm biến và lịch sử điều khiển.
- Xây dựng các REST API cho dữ liệu cảm biến, biểu đồ, trạng thái thiết bị, điều khiển đèn, lịch sử và tài khoản.
- Xây dựng giao diện Web gồm Dashboard, Data Sensor, History và Account; có thành phần bảng hỗ trợ tìm kiếm/lọc/sắp xếp/phân trang phía trình duyệt.

5.2. Hạn chế và hướng phát triển

Một số chức năng ở Frontend hiện vẫn dùng dữ liệu giả lập, chưa gọi Backend. WebSocket được mô tả trong SRS nhưng chưa xuất hiện trong source code. Do đó, hệ thống chưa thể xem là hoàn thiện toàn bộ luồng từ phần cứng đến giao diện.

- Kết nối Dashboard với GET /api/sensors/latest, GET /api/sensors/chart và GET /api/devices; thay thao tác bật/tắt mô phỏng bằng POST /api/devices/{code}/control.
- Kết nối Data Sensor và History với các API tương ứng, bổ sung phân trang/lọc ở Backend nếu dữ liệu lớn.
- Kết nối Account với GET/PUT /api/account và POST /api/account/upload; loại bỏ việc chỉ lưu localStorage.
- Triển khai WebSocket để đẩy dữ liệu và trạng thái tới giao diện, hoặc cập nhật lại SRS nếu lựa chọn cơ chế polling.
- Bổ sung xác thực người dùng, phân quyền và bảo vệ thông tin nhạy cảm trong cấu hình kết nối; không lưu mật khẩu trực tiếp trong source code.
- Bổ sung kiểm thử API bằng Postman và kiểm thử tích hợp với thiết bị thật, ghi lại kết quả đo độ trễ và tình huống mất kết nối.

Qua quá trình thực hiện, đề tài giúp củng cố kiến thức về kiến trúc ứng dụng Web, Spring Boot, JPA, MySQL và giao thức MQTT. Các phần đã triển khai là nền tảng để tiếp tục hoàn thiện khả năng tích hợp giữa giao diện, máy chủ và thiết bị IoT.

PHỤ LỤC: DANH MỤC MÃ NGUỒN

Thư mục / tệp	Vai trò
controller/AccountController.java	API thông tin tài khoản và upload PDF.
controller/DeviceController.java	API danh sách thiết bị và điều khiển bật/tắt.
controller/HistoryController.java	API truy vấn lịch sử điều khiển.
controller/SensorController.java	API trạng thái, dữ liệu mới nhất, danh sách và biểu đồ cảm biến.
entity/*.java	Các lớp ánh xạ bảng dữ liệu.
repository/*.java	Các interface truy cập dữ liệu bằng Spring Data JPA.
mqtt/MqttService.java	Kết nối MQTT, nhận bản tin, lưu dữ liệu và gửi lệnh.
static/*.html, css/style.css	Các trang giao diện và định dạng.
static/js/*.js	Logic điều hướng, Dashboard, bảng dữ liệu, lịch sử và tài khoản.

Nguồn tài liệu sử dụng: SRS_IOT(2).docx và mã nguồn src.zip do sinh viên cung cấp. Báo cáo này đã thay nội dung kỹ thuật theo dự án Smart Sensor của Lê Minh Trí; không sử dụng thông tin cá nhân, API hay công nghệ riêng của báo cáo tham khảo.